

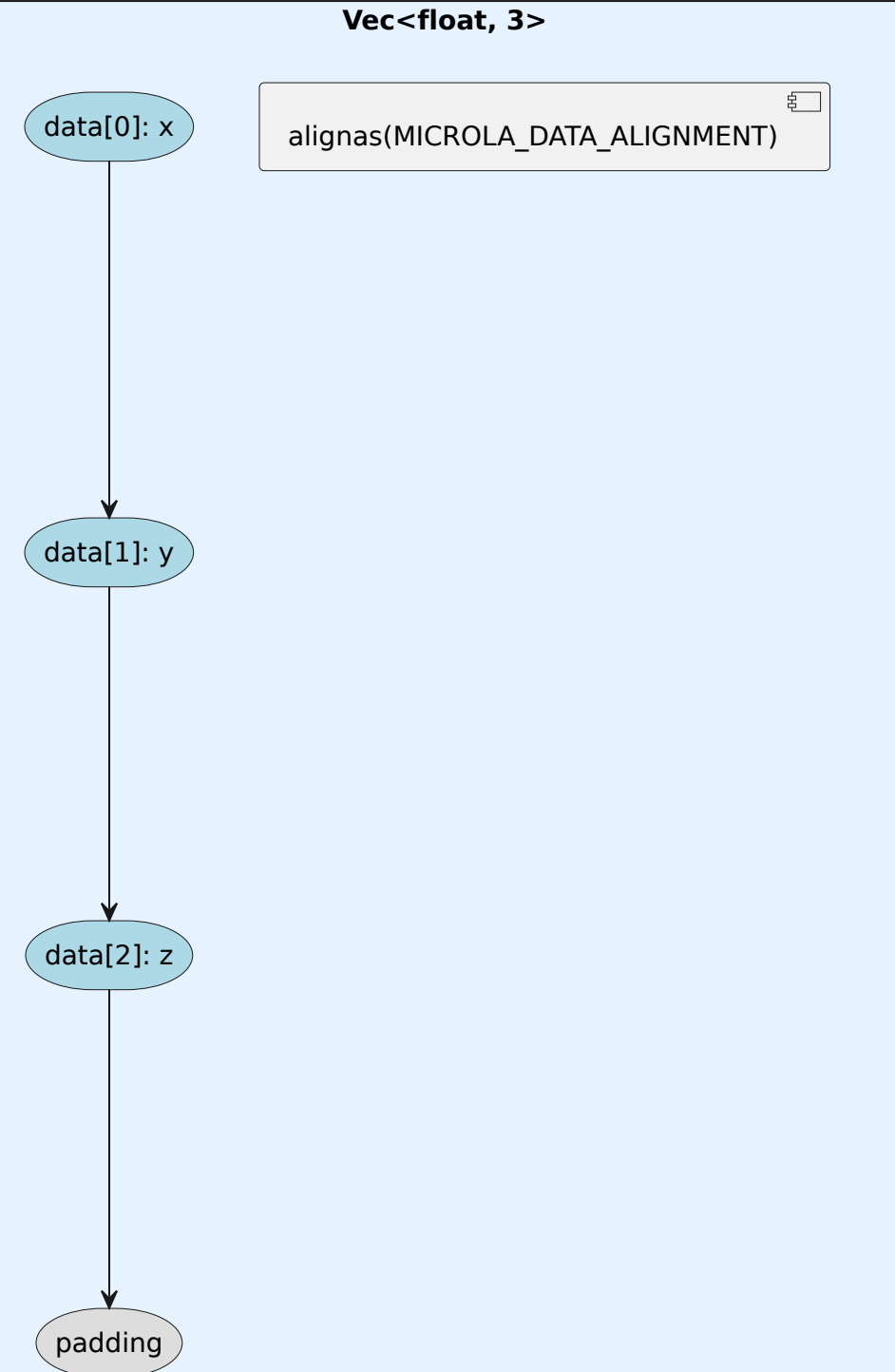
MicroLA Memory Layout & Alignment

Vec<T, N> Memory Layout

16-byte aligned
12 bytes data + 4 bytes padding
SIMD-friendly: loads as float32x4_t

Padding ensures:

- NEON vld1q_f32 compatibility
- Cache line alignment
- DMA transfer efficiency



Mat<T, R, C> Memory Layout

Row-major order:
 $M(i,j) = \text{data}[i * C + j]$

Benefits:

- Cache-friendly row access
- Natural C/C++ layout
- SIMD row operations
- Compatible with OpenGL (transpose)

Size: 36 bytes (9 floats)
Aligned to 16 bytes for SIMD



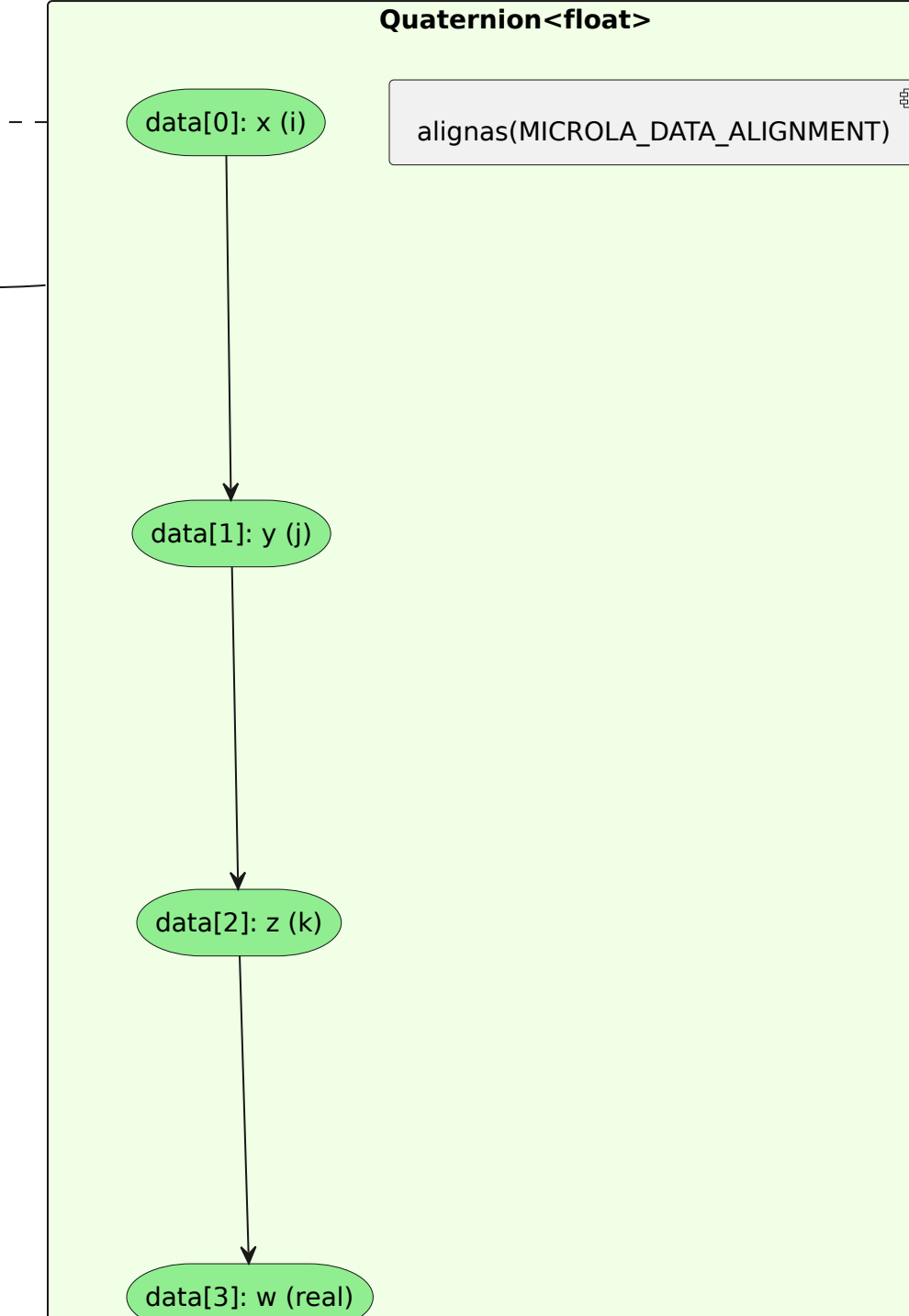
Quaternion<T> Memory Layout

[x, y, z, w] layout

Rationale:

- First 3 elements = Vec<T,3>
- Enables Vec3 SIMD operations
- Optimal for rotate(vector)
- Compatible with most libraries

Size: 16 bytes (4 floats)
Perfect SIMD register fit



Alignment Benefits

DMA Compatibility

Embedded friendly

No alignment faults

Hardware transfer ready

SIMD Performance

Aligned loads (vld1q_f32)

Aligned stores (vst1q_f32)

No unaligned penalty

Cache Efficiency

Cache line alignment

Reduced misses

Better prefetch

Memory Guarantees:

- All types: alignas(MICROLA_DATA_ALIGNMENT) — 4 B (embedded/CMSIS), 16 B (NEON/SSE), 32 B (AVX), 64 B (AVX-512)
- Standard layout (offsetof compatible)
- Contiguous memory (DMA safe)
- No virtual functions (POD-like)
- Trivially copyable (memcpy safe)

Size Validation (host default, MICROLA_DATA_ALIGNMENT=16):

```
static_assert(sizeof(Vec<float,3>) == 16)
static_assert(sizeof(Mat<float,3,3>) >= 36)
static_assert(sizeof(Quaternion<float>) == 16)
```

Embedded target (MICROLA_EMBEDDED, MICROLA_DATA_ALIGNMENT=4):

```
static_assert(sizeof(Vec<float,3>) == 12) // 4 B saved
static_assert(sizeof(Mat<float,3,3>) == 36) // 12 B saved
static_assert(sizeof(Quaternion<float>) == 16)
```